

钢铁表面缺陷检测系统 — 需求规格说明书

版本：V1.0

日期：2026-05-25

项目周期：4 周

目录

- 项目概述
- 分析阶段
- 设计阶段
- 开发阶段
- 测试阶段
- 上线部署阶段
- 运维阶段
- 附录

1. 项目概述

1.1 项目背景

钢铁生产企业当前采用人工目视 + 抽检方式进行表面缺陷检测，存在以下核心痛点：

痛点	描述
速度不匹配	人工检测节拍跟不上产线速度，成为产能瓶颈
主观性差	不同质检员标准不一，一致性难以保证
新缺陷适应慢	出现新型缺陷时人工识别培训周期长
数据孤岛	检测记录纸质化，无法进行数据分析和模型迭代

1.2 项目目标

基于 YOLO + 视觉大模型（VLM）双引擎架构，构建一套钢铁表面缺陷自动化检测系统，实现：

实时缺陷检测与分类
人工审核复核机制
统计报表与数据导出
通过 OpenClaw Agent 框架提供自然语言交互接口

1.3 项目价值

降低漏检率，减少客诉与经济损失
提升检测一致性，消除人工主观差异
积累标注数据，支持模型持续迭代
数字化检测记录，打通数据孤岛

2. 分析阶段

2.1 业务现状分析

2.1.1 当前检测流程

产线运行 → 人工目视检查 → 抽检 → 纸质记录 → 离线统计

产线速度：高速连续运行
检测方式：人工目视，抽检比例有限
记录手段：纸质表格，事后录入

2.1.2 历史缺陷数据分析

客诉来源：主要来自漏检缺陷流入下游客户
缺陷类型分布：需统计裂纹、划痕、氧化皮、压痕、气泡等典型钢铁表面缺陷
经济损失：客诉赔偿 + 退货 + 品牌影响

2.1.3 痛点量化

痛点维度	现状	目标
漏检率	人工漏检率高	显著降低
检测节拍	跟不上产线	匹配产线速度
人员流动	培训周期长	系统即装即用
数据利用	纸质记录无法分析	数字化、可追溯

2.2 需求分析

2.2.1 功能需求（FR）

编号	功能	描述	优先级
FR-01	实时检测	通过工业相机采集图像，实时进行缺陷检测	P0
FR-02	双引擎检测	YOLO 直出 + VLM 复核双引擎协同工作	P0
FR-03	人工审核	提供 Web 界面，质检员可对检测结果进行审核	P0
FR-04	统计报表	按时间、缺陷类型等维度生成统计图表	P1
FR-05	数据导出	支持 CSV 导出、Bad Case 数据集导出	P1
FR-06	自然语言交互	通过 OpenClaw Agent 支持自然语言查询	P2
FR-07	图像采集	支持 RTSP 流读取、工业相机硬触发	P0

2.2.2 非功能需求（NFR）

编号	类别	要求	指标
NFR-01	性能	单张图像推理延迟	< 200ms (YOLO)
NFR-02	准确率	检测 mAP	≥ 85% (目标值)
NFR-03	可用性	系统可用率	≥ 99.5% (生产时间)
NFR-04	安全性	数据隐私、凭证隔离	API Key 不硬编码，沙箱隔离
NFR-05	可扩展性	新增检测引擎/导出格式	模块化架构支持
NFR-06	实时性	视频流处理帧率	≥ 15 FPS

2.2.3 用户角色分析

角色	职责	核心诉求
质检员	日常操作检测系统、审核结果	界面简洁、操作便捷、响应快速
主管	查看报表、质量趋势分析	数据可视化、导出方便、多维度统计
AI 工程师	模型训练、迭代优化	训练脚本、Bad Case 收集、模型版本管理
工艺工程师	分析缺陷与工艺参数关联	缺陷统计、根因分析

2.2.4 用例分析

质检员用例：
实时检测 → 查看结果 → 人工审核 → 确认/修正 → 提交记录

主管用例：
登录 → 查看报表 → 筛选日期/缺陷类型 → 导出报告

AI 工程师用例：
收集 Bad Case → 标注 → 训练模型 → 评估 → 部署更新

工艺工程师用例：
查看缺陷统计 → 关联工艺参数 → 分析根因 → 提出改进建议

2.3 关键性能指标（KPI）

指标	定义	目标值
召回率 (Recall)	实际缺陷中被检出的比例	≥ 90%
精确率 (Precision)	检出缺陷中实际为缺陷的比例	≥ 85%
过杀率	误检为缺陷的正常产品比例	≤ 5%
推理延迟	单张图像从前处理到输出结果的时间	< 200ms
系统可用率	生产时间内正常运行比例	≥ 99.5%

2.4 约束条件

约束类型	限制
硬件	工控机 + RTX 4060 8G GPU
软件	Windows 10/11、Python 3.8+、CUDA 11.x
网络	产线内部局域网, VLM API 需外网
标注数据	初期标注数据量有限, 需借助少样本/零样本能力
周期	4 周完成知识学习 + 系统开发

2.5 风险识别与应对

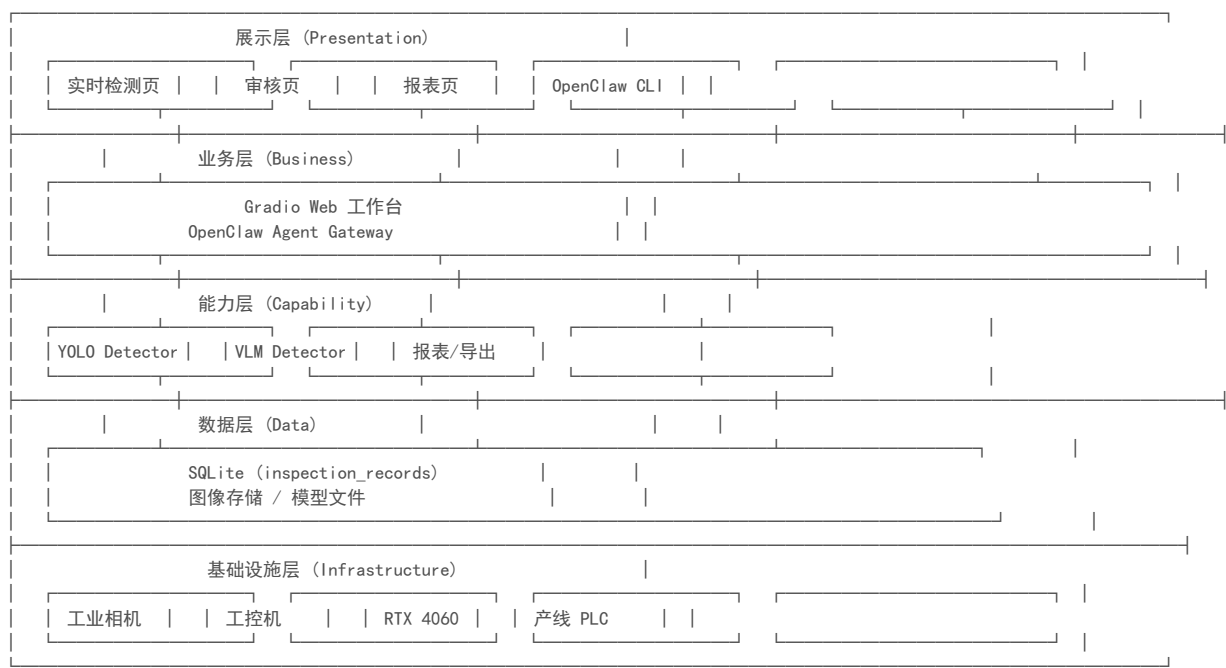
风险	影响	概率	应对措施
VLM API 不稳定	复核功能不可用	中	重试机制 + 本地 YOLO 兜底
小目标漏检	召回率低	高	提升输入分辨率 + Anchor 微调
标注数据不足	模型精度差	高	VLM 辅助标注 + 数据增强
GPU 资源受限	推理速度慢	中	TensorRT 加速 + 模型量化
环境光照干扰	误检增加	中	光源标准化 + 数据增强覆盖
产线改造配合	相机安装调试延误	低	提前协调 + 模拟环境验证

2.6 技术预研结论

技术方向	选型	理由
目标检测框架	YOLOv8/v10/v12	工业级成熟度、速度精度平衡
视觉大模型	Qwen-VL 系列	中文优化、API 稳定、性价比高
Agent 框架	OpenClaw	模块化、本地优先、自然语言接口
Web 框架	Gradio	快速原型、CV 任务友好
数据库	SQLite	轻量级、零配置、满足单机需求
推理加速	TensorRT + ONNX	显著降低延迟、充分利用 GPU

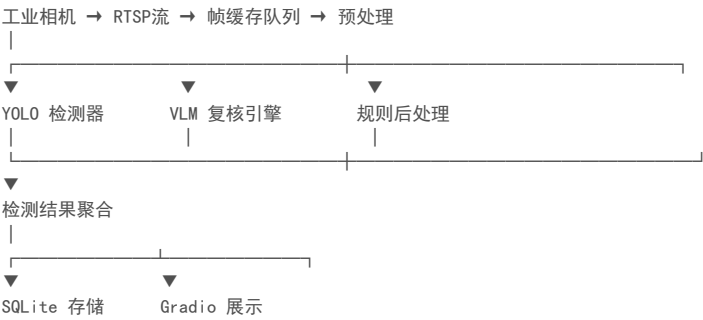
3. 设计阶段

3.1 总体架构



3.2 数据流设计

3.2.1 检测数据流



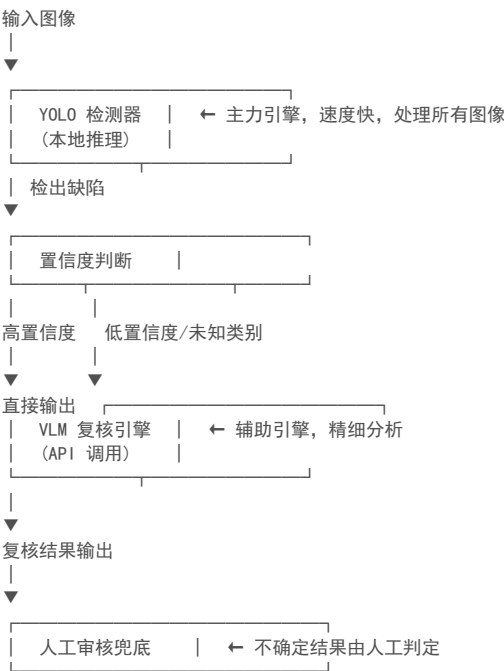
3.2.2 审核数据流

Gradio 审核页 → 质检员修正 → 更新数据库 → 同步 Bad Case 收集

3.2.3 导出数据流

数据库查询 → 数据聚合 → CSV 导出 / Bad Case 导出 / HTML 报告

3.3 双引擎协同架构



3.4 模块划分

模块	文件	职责
图像采集	camera.py	RTSP 流读取、帧缓存、硬触发集成
YOLO 检测引擎	detection_engine.py	模型加载、预处理、推理、后处理
VLM 检测引擎	vlm_engine.py	API 调用、重试机制、结果解析
数据持久化	db_manager.py	SQLite CRUD、异步写入、批量插入
Gradio 工作台	app.py	实时检测页、审核页、报表页
导出模块	exporter.py	CSV/数据集/HTML 报告导出
OpenClaw Skill	skills/yolo-detect/, skills/vlm-detect/	Agent 可调用的检测能力
训练脚本	scripts/train_yolo.py	模型训练
数据集准备	scripts/prepare_dataset.py	格式转换、数据集划分
基准测试	scripts/benchmark.py	延迟、吞吐、GPU 利用率

3.5 数据库设计

inspection_records 表

字段名	类型	说明
id	INTEGER PRIMARY KEY	自增主键
timestamp	DATETIME	检测时间
image_path	TEXT	原始图像路径

result_path	TEXT	标注结果图像路径
yolo_result	TEXT (JSON)	YOLO 检测结果
vlm_result	TEXT (JSON)	VLM 复核结果（可为空）
final_result	TEXT (JSON)	最终结果（经审核修正）
defect_types	TEXT	缺陷类型列表
defect_count	INTEGER	缺陷数量
confidence	REAL	综合置信度
reviewer	TEXT	审核人
review_status	TEXT	审核状态（pending/confirmed/corrected）
review_time	DATETIME	审核时间
note	TEXT	备注

3.6 项目代码结构

```

steel-defect-detection/
├── main.py           # 主入口
├── app.py            # Gradio Web 工作台
├── cli.py            # 命令行工具
├── config.yaml       # 配置文件
├── .env              # 环境变量 (API Key等)
├── requirements.txt  # 依赖清单
├── setup.bat / install.sh # 一键部署脚本
├──
├── src/
│   ├── camera.py      # 图像采集模块
│   ├── detection_engine.py # YOLO 检测器封装
│   ├── vlm_engine.py  # VLM 检测器封装
│   ├── base_detector.py # 检测器基类
│   ├── db_manager.py  # 数据持久化模块
│   ├── exporter.py    # 导出模块
│   └── utils/         # 工具函数
├──
├── models/           # 模型文件目录
│   └── weights/      # 预训练权重
├──
├── data/             # 数据目录
│   ├── images/       # 采集图像
│   ├── labels/       # 标注文件
│   ├── datasets/     # 数据集
│   └── exports/      # 导出文件
├──
├── skills/           # OpenClaw Skill 目录
│   ├── yolo-detect/
│   │   ├── SKILL.md
│   │   └── detect.py
│   └── vlm-detect/
│       ├── SKILL.md
│       └── detect.py
├──
├── scripts/          # 自动化脚本
│   ├── train_yolo.py  # 训练脚本
│   ├── prepare_dataset.py # 数据集准备
│   ├── benchmark.py   # 性能基准测试
│   └── log_analyzer.py # 日志分析
├──
├── tests/            # 测试目录
│   ├── unit/
│   ├── integration/
│   └── fixtures/
├──
└── docs/             # 文档
    ├── 需求规格说明书.md
    ├── 用户手册.md
    └── API文档.md
    
```

3.7 OpenClaw 集成设计

3.7.1 集成原因

- 提供自然语言交互接口，降低操作门槛
- 模块化 Skill 设计，检测能力可复用
- 本地优先架构，与工控机部署匹配
- 长短期记忆，支持上下文连续对话

3.7.2 Skill 设计

Skill	功能	输入	输出
yolo-detect	调用 YOLO 模型进行缺陷检测	图像路径	检测结果 JSON
vlm-detect	调用 VLM API 进行精细分析	图像路径 + Prompt	分析结果 JSON

3.7.3 Agent 设计

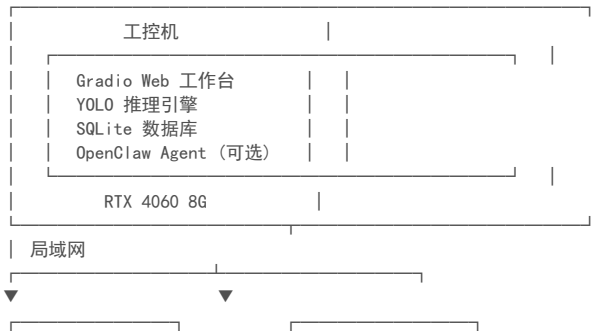
- 名称: detect-analyst ext
- 能力: 组合 yolo-detect 和 vlm-detect 两个 Skill
- 推理模式: ReAct (思考→行动→观察)
- 典型对话:
 - "检测这张钢板的表面缺陷"
 - "分析最近一小时的缺陷趋势"
 - "导出今天的检测报告"

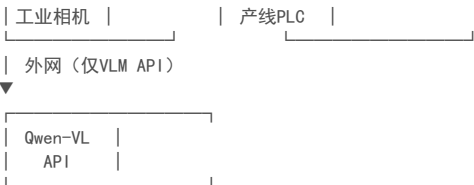
3.7.4 与 Gradio 的关系

维度	Gradio 工作台	OpenClaw Agent
定位	主力操作界面	辅助/高级交互
用户	质检员、主管	AI 工程师、主管
阶段	当前主力	过渡探索、未来方向

3.8 部署架构

3.8.1 单机部署（主力方案）





3.8.2 弹性扩展（未来）

- 多工位部署时可增加推理节点
- 云端做模型训练与 VLM API 调用
- 边缘端做实时推理
- 中心数据库汇总多产线数据

4. 开发阶段

4.1 开发环境

类别	规格
硬件	工控机、NVIDIA RTX 4060 8G、工业相机
操作系统	Windows 10/11
Python	3.8+
CUDA	11.x
核心依赖	PyTorch、Ultralytics、OpenCV、Gradio、SQLite3
版本控制	Git + GitLab
协作方式	分支开发、Code Review

4.2 开发计划（2 周）

第 1 周：核心功能

天数	任务	产出
D1-D2	项目脚手架搭建、环境配置、目录初始化	可运行的空项目
D2-D3	图像采集模块（RTSP 流 + 帧缓存）	camera.py
D3-D5	YOLO 检测引擎（加载、推理、后处理）	detection_engine.py
D5-D6	Gradio 实时检测页面	app.py（检测页）
D6-D7	SQLite 数据库 + 数据持久化	db_manager.py

第 2 周：完整系统

天数	任务	产出
D8-D9	VLM 检测引擎（API 调用 + 容错）	vlm_engine.py
D9-D10	双引擎协同 + 人工审核页面	审核页、协同逻辑
D10-D11	统计报表 + 导出模块	报表页、exporter.py
D11-D12	OpenClaw Skill 开发	yolo-detect + vlm-detect Skill
D12-D13	训练/数据脚本 + 基准测试	scripts/ 下各脚本
D13-D14	联调、Bug 修复、文档	完整可交付系统

4.3 核心模块实现要点

4.3.1 图像采集模块

核心设计：多线程缓冲队列避免帧丢失
 # RTSP → 独立采集线程 → Queue → 主线程消费
 # 支持 PLC 硬触发信号集成

技术要点：

OpenCV VideoCapture + RTSP 协议
 线程安全的环形缓冲区队列
 丢帧策略：取最新帧，跳过积压帧
 PLC 信号通过串口/GPIO 接入

4.3.2 YOLO 检测引擎

```
# 统一接口设计
class BaseDetector(ABC):
    @abstractmethod
    def detect(self, image, **kwargs) -> DetectionResult:
        pass

class YOLODetector(BaseDetector):
    def __init__(self, model_path, conf_threshold=0.25):
        self.model = YOLO(model_path)

    def detect(self, image, **kwargs) -> DetectionResult:
        results = self.model(image, conf=kwargs.get('conf', 0.25))
        return self._parse_results(results)
```

技术要点：

Ultralytics YOLO 封装（支持 v8/v10/v12）
 支持 ONNX/TensorRT 导出加速
 后处理：NMS、坐标转换、类别映射
 小目标优化：提高输入分辨率、微调 Anchor

4.3.3 VLM 检测引擎

```
class VLMDetector(BaseDetector):
    def detect(self, image, **kwargs) -> DetectionResult:
        # 图像编码 → API 调用 → JSON Schema 校验 → 重试
        pass
```

技术要点：

- Qwen-VL API 调用，Base64 图像编码
- System Prompt 工程：角色扮演 + 任务分解 + 输出约束
- JSON Schema 校验确保输出格式稳定
- 指数退避重试机制
- 超时处理（本地 YOLO 结果兜底）

4.3.4 数据库模块

技术要点：

- SQLite WAL 模式提升并发性能
- 连接池管理（sqlite3 + 线程锁）
- 异步批量写入队列
- 索引优化：timestamp、defect_types、review_status ext

4.3.5 OpenClaw Skill 开发

yolo-detect SKILL.md 示例：

```
---
name: yolo-detect
description: 使用 YOLO 模型检测钢铁表面缺陷
permissions:
- file:read
- network:local
inputs:
image_path: string # 待检测图像路径
outputs:
detections: array # 检测到的缺陷列表
---
```

vlm-detect 安全沙箱：

- 仅允许 network 权限（调用远程 API）
- 禁止文件系统写入
- 禁止执行系统命令

4.4 配置管理

config.yaml 示例

```
models:
yolo:
path: "models/weights/yolov8n-steel.pt"
conf_threshold: 0.25
iou_threshold: 0.45
device: "cuda:0"

vlm:
provider: "qwen"
model: "qwen-vl-plus"
max_retries: 3
timeout: 10

camera:
rtsp_url: "rtsp://192.168.1.100:554/stream"
buffer_size: 30
fps: 15
```

```
database:
path: "data/inspection.db"
wal_mode: true

ui:
host: "0.0.0.0"
port: 7860
share: false
```

4.5 开发过程问题预案

预期问题	解决方案
Gradio 与 OpenClaw 端口冲突	修改默认端口，分离部署
VLM API 返回格式不稳定	JSON Schema 校验 + 容错解析
SQLite 并发写入锁	WAL 模式 + 连接池 + 写队列
YOLO 小目标漏检	提升分辨率 + Anchor 微调 + 多尺度训练

5. 测试阶段

5.1 测试策略

单元测试 → 集成测试 → 系统测试 → 性能测试 → UAT → 上线

5.2 单元测试

测试模块	测试内容	工具
camera.py	帧读取、缓冲区、硬触发模拟	pytest + mock
detection_engine.py	模型加载、预处理、推理结果格式	pytest
vlm_engine.py	API 调用参数、重试逻辑、结果解析	pytest + mock
db_manager.py	CRUD 操作、并发写入、批量插入	pytest
exporter.py	CSV 格式、字段映射、编码处理	pytest
base_detector.py	接口契约、子类实现约束	pytest

覆盖率目标：核心模块 ≥ 80%

5.3 集成测试

场景	测试内容
采集→检测→存储	端到端数据流验证
双引擎协同	YOLO 直出 + VLM 复核 + 人工兜底全链路
审核→数据库更新	审核操作后的数据一致性
OpenClaw Skill 调用	Agent 调用 Skill 的完整流程
导出全链路	数据库查询→文件生成→格式验证

5.4 性能测试

测试项	方法	通过标准
推理延迟	benchmark.py 批量测试	P50 < 150ms, P99 < 300ms
吞吐量	持续压测 30 分钟	≥ 15 FPS 稳定运行
GPU 利用率	nvidia-smi 监控	推理时 ≥ 80%
内存占用	长时间运行监控	无内存泄漏, < 4GB
VLM API 延迟	统计调用耗时	P95 < 5s

5.5 用户验收测试（UAT）

场景	验收标准
实时检测	质检员可流畅查看视频流检测结果
人工审核	可对检测结果进行修正、确认、提交
报表查看	可按日期/类型筛选并查看统计图表
数据导出	可正常导出 CSV、Bad Case 数据集、HTML 报告
异常处理	相机断连、API 超时系统不崩溃，有友好提示
开机自启	工控机重启后系统自动启动运行

5.6 测试数据

- 准备 ≥ 200 张真实产线图像（覆盖各缺陷类型）
- 准备 ≥ 50 张无缺陷正常图像
- 标注质量经人工交叉验证
- 包含边界 Case：小目标、重叠缺陷、光照变化

6. 上线部署阶段

6.1 部署方案

6.1.1 工控机单机部署

- 部署步骤：
- 1. 环境准备
 - 安装 Python 3.8+、CUDA 11.x、cuDNN
 - 安装工业相机驱动与 SDK
 - 配置 RTSP 流地址
 - 2. 系统部署
 - 执行 setup.bat（自动创建虚拟环境、安装依赖）
 - 放置模型权重文件至 models/weights/
 - 配置 config.yaml 和 .env
 - 3. 启动验证
 - 运行 cli.py verify 验证各模块状态
 - 运行 benchmark.py 确认性能达标
 - 试运行 30 分钟无异常
 - 4. 开机自启
 - Windows：配置任务计划程序 / NSSM 服务
 - 设置系统自动登录
 - 配置 watchdog 进程监控

6.1.2 服务化部署

```
# 一键部署
./setup.bat

# 启动服务（开机自启配置后自动执行）
python main.py --mode server

# 命令行验证
python cli.py verify
```

6.2 上线 Checklist

序号	检查项	状态
1	所有单元测试通过	
2	集成测试通过	
3	性能测试达标	
4	UAT 验收通过	
5	模型权重文件就位	
6	配置文件正确填充	
7	API Key 已配置且安全存储	
8	相机连接正常	
9	数据库初始化完成	
10	开机自启配置完成	
11	监报告警配置完成	
12	用户手册已交付	
13	运维文档已交付	
14	回滚方案就绪	

6.3 灰度发布策略

- 1. 离线验证（1天）

- 在生产环境镜像中跑历史数据
- 2. 并跑验证（3天）
 - 系统检测 + 人工继续检测，对比结果但不替代
- 3. 正式切换
 - 系统为主，人工抽检复核
- 4. 持续优化
 - 收集 Bad Case → 模型迭代 → 增量更新

6.4 回滚方案

场景	回滚操作
模型精度不达标	切回上一版本模型权重
系统崩溃	切回人工检测流程，修复后重新部署
API 不可用	VLM 引擎降级，仅用 YOLO 继续工作

7. 运维阶段

7.1 监控体系

7.1.1 系统监控指标

指标	监控方式	告警阈值
CPU 使用率	Windows 性能计数器	> 90% 持续 5 分钟
GPU 使用率	nvidia-smi	< 50%（推理时异常低）
GPU 温度	nvidia-smi	> 85° C
内存占用	psutil	> 80%
磁盘空间	psutil	< 10GB 剩余
推理延迟	内置统计	P99 > 500ms
检测帧率	内置统计	< 10 FPS
相机在线状态	心跳检测	断连立即告警
VLM API 可用性	健康检查	连续 3 次失败告警
数据库大小	文件系统监控	> 1GB（需归档）

7.1.2 告警方式

- 工控机本地弹窗提示
- Gradio 页面状态指示灯（绿/黄/红）
- 邮件/企业微信通知（可选）

7.2 日志管理

7.2.1 日志分类

日志类型	内容	保留周期
系统日志	启动/停止/异常/配置变更	30 天
检测日志	每次检测的详细结果	90 天
API 调用日志	VLM 请求/响应/耗时	30 天
性能日志	延迟、FPS、资源占用	14 天

7.2.2 日志分析

log_analyzer.py 脚本：每日自动生成简要报告

异常检测：检测日志异常模式自动标记

趋势分析：按周/月统计检测量、缺陷分布

7.3 模型迭代优化

7.3.1 迭代流程

生产运行 → 收集 Bad Case → 人工标注 → 加入训练集 → 重新训练 → 评估 → 灰度上线

7.3.2 自动化机制

Bad Case 自动导出脚本：每日/每周自动导出低置信度 + 审核修正记录

标注辅助：VLM 预标注 + 人工修正

模型版本管理：每次训练生成带时间戳的模型包（权重 + 标签映射 + 配置文件）

灰度更新脚本：支持增量替换，保留上一版本备份

7.4 数据运维

操作	频率	说明
数据库备份	每日	自动 SQLite dump
图像归档	每周	超过 30 天的图像迁移至外部存储
数据库清理	每月	删除超过 90 天的检测日志（可选）
模型权重备份	每次更新	保留历史版本

7.5 日常运维操作

操作	命令	说明
启动系统	python main.py --mode server	启动 Web 工作台
停止系统	Ctrl+C 或任务管理器	优雅关闭
查看状态	python cli.py status	检查各模块运行状态
验证环境	python cli.py verify	验证相机/模型/数据库

切换模型	<code>python cli.py switch-model <path></code>	切换模型权重
导出报告	<code>python cli.py export --date 2026-05-25</code>	命令行导出
查看日志	<code>python log_analyzer.py --today</code>	查看今日日志摘要

7.6 常见问题处理

问题	排查步骤	解决方案
相机断连	1. 检查物理连接 2. ping 相机 IP 3. VLC 测试	重启相机 / 检查网线 / 联系硬件
推理变慢	1. nvidia-smi 2. 任务管理器 3. 系统日志	重启系统 / 检查是否有其他 GPU 进程
VLM API 超时	1. ping 外网 2. 检查 API Key 3. 检查配额	降级至纯 YOLO 模式 / 联系 API 提供商
磁盘空间不足	1. 检查 data/images/ 2. 检查日志大小	执行归档脚本 / 手动清理过期数据
误检率升高	1. 对比历史数据 2. 检查光源 3. 检查相机	调整光源 / 收集 Bad Case 重新训练

7.7 持续改进计划

阶段	时间	目标
稳定运行	上线后 1-2 周	监控稳定性、收集反馈
快速迭代	上线后 3-4 周	基于 Bad Case 微调模型
功能增强	上线后 2-3 月	VLM 能力深化、报表增强
规模扩展	上线后 3-6 月	多产线部署、云端训练、中心数据库

8. 附录

8.1 技术选型对比

维度	YOLO 方案	VLM 方案	双引擎方案（选型）
推理速度	★★★★★	★★☆	★★★★☆
检测精度	★★★★☆	★★★★☆	★★★★★
新缺陷适应	★★☆	★★★★★	★★★★★
部署成本	低（本地）	中（API 费用）	中
可解释性	★★★☆☆	★★★★☆	★★★★☆
稳定性	★★★★★	★★★★☆	★★★★☆

8.2 术语表

术语	说明
YOLO	You Only Look Once, 单阶段目标检测算法
VLM	Vision Language Model, 视觉语言大模型
OpenClaw	开源 AI Agent 框架, NL2Sys 理念
Gradio	Python Web UI 框架, 适合 ML 应用
RTSP	Real Time Streaming Protocol, 实时流传输协议
PLC	Programmable Logic Controller, 可编程逻辑控制器
IoU	Intersection over Union, 交并比
mAP	mean Average Precision, 平均精度均值
NMS	Non-Maximum Suppression, 非极大值抑制
Bad Case	模型预测错误的样本
RAG	Retrieval-Augmented Generation, 检索增强生成

8.3 参考资料

- Ultralytics YOLO 官方文档
- Qwen-VL API 文档
- OpenClaw 框架文档
- 《计算机视觉与工业质检基础》
- 《大模型基础知识》

文档审批

角色	姓名	日期	签名
编写	—	2026-05-25	
审核	—		
批准	—		